COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	08/04/2026	**DATE SUBMITTED:**	08/11/2026
NAME:	Lance Cristopher S. Salcedo	**INSTRUCTOR:**	Engr. Jimlord Quejardo

HANDS ON ACTIVITY NO. 5.1 **Queues**

1. PROCEDURE OUTPUT

ILO A

Code:

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    //create an object
    std::queue<char> myQ;

    myQ.push('L');
    myQ.push('A');
    myQ.push('N');
    myQ.push('C');
    myQ.push('E');

    //displays queue
    display(myQ);

    //try all member functions:
    //empty
    if (myQ.empty()){
        std::cout << "The queue is empty. " << std::endl;
    }else{
        std::cout << "The queue is not empty." << std::endl;
    }
    //size
    std::cout << "The size of the queue is: " << myQ.size() << std::endl;

    //front
    std::cout << "The front of the queue: " << myQ.front();
    display(myQ);

    //back
    std::cout << "The back of the queue: " << myQ.back();
    display(myQ);

    //pop
    myQ.pop();
    std::cout <<"The popped queue: ";
```

```
40      display(myQ);
41
42      //push
43      myQ.push('L');
44      std::cout << "The pushed queue: ";
45      display(myQ);
46
47
48      //display all
49      display(myQ);
50
51      return  0;
52  }
53
54  void display(std::queue<char> copyQ){
55      //create a copy of the queue that was passed
56      std::queue<char> c = copyQ; //temp queue
57
58      while(!c.empty()){
59          std::cout<<" " << c.front();
60          c.pop();
61      }
62      std::cout << "\n";
63  }
```

Output:

```
 L A N C E
The queue is not empty.
The size of the queue is: 5
The front of the queue: L L A N C E
The back of the queue: E L A N C E
The popped queue:  A N C E
The pushed queue:  A N C E L
 A N C E L
```

Observations:

From what I observed, the queue reminds me of the previous activities like stacks and arrays. Compared to stacks that use LIFO (last in, first out), queue uses FIFO (first in, first out). Similar to stacks, it also has operations. I also noticed how the queue is arranged compared to stacks.

ILO B

Code:

queue_LL.h:

```
#ifndef QUEUE_LL_H
#define QUEUE_LL_H

//create a class the node
template <typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

//create the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

//enqueue operation
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //creates the new node*
    qNode<T> *newNode = new_node(newData);
    //inserting into an empty queue
    if((*frontPtr)==nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }
    //inserting an item into a non empty queue
    (*backPtr)->next = newNode;
```

```
33      (*backPtr) = newNode;
34  }
35
36  template <typename T>
37  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
38      //create a temporary node
39      qNode<T> *deleteNode = nullptr; //temp variable
40      deleteNode = (*frontPtr);
41
42      //check if the queue only has 1 node
43      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
44          (*frontPtr) = nullptr;
45          (*backPtr) = nullptr;
46          delete deleteNode;
47          return;
48      }
49
50      //add a check if the queue is already empty
51      if((*frontPtr) == nullptr && (*backPtr) == nullptr){
52          std::cout << "The queue is empty." << std::endl;
53      }
54
55      //delete node
56      (*frontPtr) = (*frontPtr)->next;
57      deleteNode->next = nullptr;
58      delete deleteNode;
59  }
60  //check the front
61  template <typename T>
62  void qFront(qNode<T>** frontPtr){
```

```
63      //checker
64      if((*frontPtr) != nullptr){
65      std::cout << "The front of the queue: " << (*frontPtr)->data
66      << std::endl;
67      }
68  }
69  //check the back
70  template <typename T>
71  void qBack(qNode<T>** backPtr){
72      //checker
73      if((*backPtr) == nullptr){
74          std::cout << "The queue is empty." << std::endl;
75          return;
76      }
77      std::cout << "The back of the queue: " << (* backPtr)->data
78      << std::endl;
79      return;
80  }
81  //display all
82  template <typename T>
83  void qDisplayAll(qNode<T> **frontPtr, qNode<T>** backPtr){
84      //checker
85      if((*frontPtr) == nullptr && (*backPtr) == nullptr){
86          std::cout << "The queue is empty." << std::endl;
87          return;
88      }
89      //traverse
90      while ((frontPtr) != nullptr){
91          std::cout << (*frontPtr)->data << " ";
92          (*frontPtr) = (*frontPtr)->next;
93      }
94      std::cout << std::endl;
95  }
96  #endif
```

main2.cpp:

```cpp
#include <iostream>
#include "queue_LL.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    //enqueue
    enqueue('L', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('A', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('N', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('C', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    enqueue('E', &front, &back);
    std::cout << front->data << " " << back->data << std::endl;

    //dequeue
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;

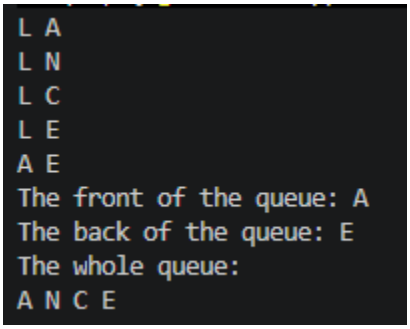
    //check for front
    qFront(&front);

    //check for back
    qBack(&back);

    //displaying for all
    std::cout << "The whole queue: " << std::endl;
    qDisplayAll(&front, &back);

    return 0;
}
```

Output:



Observations:

This queue has a linked list. It also reminds me of other activities with linked lists. It also uses operations with the addition of enqueue (adds the element to the rear (back)) and dequeue (removes an element in the front). It is also a bit confusing to track when making this code since it also requires a parenthesis for every frontPtr and backPtr.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

printQArr.h:

```h
#ifndef PRINTQARR_H
#define PRINTQARR_H
#include <iostream>

class job{
    private:
        int id;
        std::string name;
        int pageCount;
    public:
        //default constructor for arrays
        job(): id(0), name(""), pageCount(0){}
        job(int jobId, std::string jobName, int printPages){
            id = jobId;
            name = jobName;
            pageCount = printPages;
        };
        //printing details
    void printJobDetails(){
        std::cout
        << "\nID: " << this->id
        << "\nName: "  << this->name
        << "\nPage Count: "  << this->pageCount << std::endl;
    };
};

```

```
27  template <typename J>
28  class jobQueueArr{
29      private:
30          J* j_array;
31          size_t j_capacity;
32          size_t j_size;
33          int j_front;
34          int j_back;
35      public:
36          jobQueueArr(size_t capacity);
37          ~jobQueueArr();
38          bool Empty();
39          bool Full();
40
41          size_t Size();
42
43          void Clear();
44
45          J Front();
46          J Back();
47
48          void Enqueue(J newData);
49          void Dequeue();
50
51          jobQueueArr(const jobQueueArr& other);
52          jobQueueArr& operator = (const jobQueueArr& other);
53  };
54
55  template <typename J>
56  jobQueueArr<J>::jobQueueArr(size_t capacity){
57      //define the capacity of the array;
```

```
58      j_capacity = capacity;
59      //set the size to 0
60      j_size = 0;
61      //set the front and back to 0 and -1 respectively
62      j_front = 0;
63      j_back = -1;
64      //allocate a new space for the j_array that was created
65      j_array = new J[j_capacity];
66      std::cout <<"Constructor is called." << std::endl;
67  }
68
69  template <typename J>
70  jobQueueArr<J>::~jobQueueArr(){
71      delete[] j_array;
72      std::cout << "Deconstructor was Called." << std::endl;
73  }
74
75  template <typename J>
76  bool jobQueueArr<J>::Empty(){
77      return j_size == 0;
78  }
79
80  template <typename J>
81  bool jobQueueArr<J>::Full(){
82      return j_size == j_capacity;
83  }
84
85  template <typename J>
86  size_t jobQueueArr<J>::Size(){
87      return j_size;
```

```
88  }
89
90  template <typename J>
91  void jobQueueArr<J>::Clear(){
92      //queue size to 0
93      j_size = 0;
94      //restores j_front and j_back to their initial values
95      j_front = 0;
96      j_back = -1;
97      return;
98  }
99
100 template <typename J>
101 J jobQueueArr<J>::Front(){
102     //check if the queue is not empty
103     if(Empty()){
104         std::cout << "The queue is empty." << std::endl;
105     }
106     //return the front of the queue
107     return j_array[j_front];
108 }
109
110 template <typename J>
111 J jobQueueArr<J>::Back(){
112     //check if the queue is not empty
113     if(Empty()){
114         std::cout << "The queue is empty." << std::endl;
115     }
116     //return the back of the queue
117     return j_array[j_back];
118 }
```

```
119
120 template <typename J>
121 void jobQueueArr<J>::Enqueue(J newData){
122     //check if full
123     if(Full()){
124         std::cout << "The queue is full." << std::endl;
125         return;
126     }
127     //computer for the back index
128     j_back = (j_back + 1) % j_capacity;
129     //store the newData to the back of the queue
130     j_array[j_back] = newData;
131     //increment size
132     j_size++;
133 }
134
135 template <typename J>
136 void jobQueueArr<J>::Dequeue(){
137     if(Empty()){
138         std::cout << "The queue is empty." << std::endl;
139         return;
140     }
141     //compute for the back index
142     j_front = (j_front + 1) % j_capacity;
143     //decrement size;
144     j_size--;
145     //what will happen if the queue is empty?
146     if (j_size == 0){
147         Clear();
148         return;
149     }
```

```
150 }
151
152 class Printer{
153     private:
154         jobQueueArr<job> print;
155     public:
156         //default constructor for arrays
157         Printer(int capacity): print(capacity){}
158         //adding print
159         void printing(job job){
160             if (print.Full()){ // if full
161                 std::cout << "Printer is full. Don't add." << std::endl;
162             }
163             print.Enqueue(job); //enqueue
164             std::cout << "Employee's queue is now added to the printer."
165             << std::endl;
166         }
167         //next print
168         void nextPrint(){
169             if (print.Empty()){
170                 std::cout << "Printer is empty. Please add."
171                 << std::endl;
172                 return;
173             }
174             print.Dequeue(); //dequeue
175             std::cout << "Print is success. Next in line please."
176             << std::endl;
177         }
178 };
179 #endif
```

printQArr.cpp:

```cpp
#include <iostream>
#include "printQArr.h"

int main(){
    //creates an intenger queue
    jobQueueArr<int> j(4);

    job j1(001, "Rudolf", 5);
    job j2(002, "Teio", 2);
    job j3(003, "Tsuyu", 3);
    job j4(004, "Kris", 6);

    Printer print(3);

    //calling enqueue, printing details, add, front, back and the size
    j.Enqueue(1);
    j1.printJobDetails();
    print.printing(j1);
    print.nextPrint();
    std::cout <<"Front: " << j.Front() << " Back: "
    << j.Back() << " Size: " << j.Size() << std::endl;
    std::cout << "\n";

    j.Enqueue(2);
    j2.printJobDetails();
    print.printing(j2);
    print.nextPrint();
    std::cout <<"Front: " << j.Front() << " Back: "
    << j.Back() << " Size: " << j.Size() << std::endl;
    std::cout << "\n";

    j.Enqueue(3);
    j3.printJobDetails();
    print.printing(j3);
    print.nextPrint();
    std::cout <<"Front: " << j.Front() << " Back: "
    << j.Back() << " Size: " << j.Size() << std::endl;
    std::cout << "\n";

    j.Enqueue(4);
    j4.printJobDetails();
    print.printing(j4);
    print.nextPrint();
    std::cout <<"Front: " << j.Front() << " Back: "
    << j.Back() << " Size: " << j.Size() << std::endl;
    std::cout << "\n";

    //dequeue because if we combine it with enqueue the size will be 1
    j.Dequeue();
    j.Dequeue();
    j.Dequeue();
    j.Dequeue();

    return 0;
}
```

Output:

```
Constructor is called.
Constructor is called.

ID: 1
Name: Rudolf
Page Count: 5
Employee's queue is now added to the printer.
Print is success. Next in line please.
Front: 1 Back: 1 Size: 1


ID: 2
Name: Teio
Page Count: 2
Employee's queue is now added to the printer.
Print is success. Next in line please.
Front: 1 Back: 2 Size: 2
```

```
ID: 3
Name: Tsuyu
Page Count: 3
Employee's queue is now added to the printer.
Print is success. Next in line please.
Front: 1 Back: 3 Size: 3


ID: 4
Name: Kris
Page Count: 6
Employee's queue is now added to the printer.
Print is success. Next in line please.
Front: 1 Back: 4 Size: 4

Deconstructor was Called.
Deconstructor was Called.
```

Analysis:

For printQArr.h:
Asides from the header files (#ifndef, #define, #endif and include <iostream>), according to the instruction I must first make two classes which are the class Job and class Printer. In the class Job, I used the access modifiers, private and public, for the private, it contains the details of the job employee which are the ID, name and the pages that they will print, in the public, I used a default constructor for arrays (job(): id(0), name(""), pageCount(0){}) then assign their variables, then later added the print details. Next is adding another class which is the jobQueueArr and its template which is J. This way we can use the queue from the lesson earlier. In the private access modifier, I put the j_array, j_capacity, j_size, j_front and j_back. In the public access modifier, I put the constructor (jobQueueArr(size_t capacity);), deconstructor (~jobQueueArr();), if its empty or full, the size, clearing, checking on what is the front and the back, and enqueue and dequeue. I also included the operations on how each function works. In the printer function, in the private access modifier, I used jobQueueArr<job> print, this will actually hold and make a constructor and deconstructor, in the output later both of them are doubled. In the public access modifier, similar to the class job, I used a default constructor Printer(int capacity): print(capacity){}). I added the function printing and nextPrint, in the function printing, it will check if its full, otherwise, it will proceed to enqueue and in the in the function nextPrint, it will check if the function printing is empty or the enqueue, otherwise, it will proceed to dequeue.

For printQArr.cpp:
In the printQArr.cpp (main file), I added the typical header files include <iostream> and the gch file (printQArr.h) to access it. In the main function, I added the template for the integer queue (jobQueueArr <int> j(4). I used j as an object to print the id, name and pages that will be printed. I then now added the function printer with print as the object. Now I call the enqueue, print details, add, front, back and the size. Lastly the dequeue, I called it before it made it to return 0 because the size of everybody who is printing will remain to 1 if it doesn't put in the end.

Conclusion:

I was able to finish the activity with a queue array instead of the linked lists since I was planning to submit it using the linked list first. I used arrays here because it's a bit easier than linked lists and doesn't complicate the code. I wish I could have improved it a bit better if who is in the front since it didn't identify who was in the front. Overall, at least I know how queues work and always identify that it is FIFO (or first in, first out).

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Gemini to explain on why I used**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.